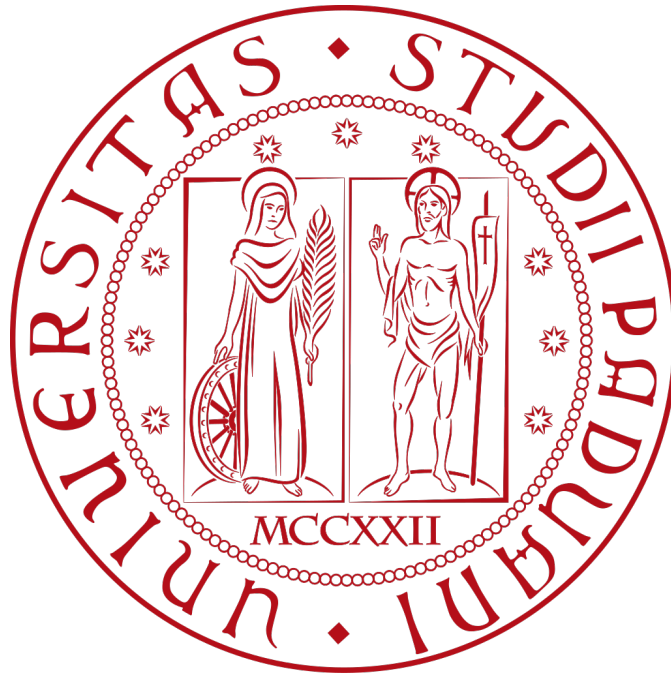




ZUCCHETTI



AI TrAIning

Norme di progetto

Filippo Sabbadin

08 / 08 / 2025

Indice

1. Introduzione	1
1.1. Scopo del documento	1
1.2. Scopo del prodotto	1
1.3. Glossario	1
1.4. Riferimenti	1
1.4.1. Riferimenti normativi	1
1.4.2. Riferimenti informativi	2
2. Processi primari	3
2.1. Processo di fornitura	3
2.1.1. Scopo e descrizione	3
2.1.2. Attività	3
2.1.3. Rapporto con il tutor aziendale	3
2.1.4. Documentazione	4
2.2. Strumenti utilizzati	4
2.3. Sviluppo	4
2.3.1. Scopo e descrizione	4
2.3.2. Attività	5
2.3.2.1. Analisi dei requisiti	5
2.3.2.2. Progettazione	6
2.3.2.3. Codifica	6
2.3.2.4. Modellazione	6
2.3.2.5. Testing	6
2.3.3. Regole di sviluppo	7
2.3.4. Strumenti usati	7
3. Processi di supporto	9
3.1. Documentazione	9
3.1.1. Scopo e descrizione	9
3.1.2. Attività	9
3.1.3. Struttura dei documenti	9
3.1.3.1. Intestazione	9
3.1.3.2. Indice	10
3.1.3.3. Corpo del documento	10
3.1.4. Documenti del progetto	10
3.1.5. Elenchi puntati	10
3.1.6. Diagrammi Use Case	10
3.1.7. Composizione tipografica	10
3.1.8. Strumenti	11
3.2. Gestione della configurazione	11

3.2.1. Scopo e descrizione	11
3.2.2. Repository	11
3.3. Gestione della qualità	11
3.3.1. Scopo e descrizione	11
3.3.2. Attività	12
3.4. Verifica	12
3.4.1. Scopo e descrizione	12
3.4.2. Strumenti	12
3.4.3. Tipi di verifica	12
3.4.3.1. Analisi statica	12
3.4.3.2. Analisi dinamica	13
3.4.4. Denominazione dei test	13
3.4.5. Verifica della documentazione	13
3.5. Validazione	14
3.5.1. Scopo e descrizione	14
4. Processi organizzativi	15
4.1. Gestione dei processi	15
4.1.1. Scopo e descrizione	15
4.1.2. Attività	15
4.1.3. Strumenti usati	15
4.2. Miglioramento	15
4.2.1. Scopo e descrizione	15
5. Metriche e standard per la qualità	16
5.1. Funzionalità	16
5.2. Performance	16
5.3. Affidabilità	17
5.4. Sicurezza	17
5.5. Usabilità	17
5.6. Portabilità	18
5.7. Manutenibilità	18

1. Introduzione

1.1. Scopo del documento

Il presente documento ha lo scopo di definire le norme di progetto che verranno seguite durante lo sviluppo del videogioco.

Le norme di progetto sono un insieme di regole e linee guida che devono essere seguite durante lo sviluppo del progetto, al fine di garantire la qualità del prodotto finale e la sua manutenibilità nel tempo.

Esse comprendono le convenzioni di codifica, le linee guida per la documentazione, le procedure di test e le pratiche di gestione del progetto.

1.2. Scopo del prodotto

Il progetto consiste nello sviluppo di un videogioco con il motore di gioco [Godot*](#) di tipo [platformer*](#), in cui il giocatore controlla un personaggio che si muove in un ambiente tridimensionale.

Il gioco si basa su temi di [Intelligenza Artificiale*](#) e [Machine Learning*](#), sviluppando meccaniche ed elementi del livello basati su questi argomenti.

Il gioco comprende 3 livelli, ognuno con un tema diverso, un menu principale che viene visualizzato non appena il gioco viene avviato, e un menu di pausa che può essere visualizzato in qualsiasi momento durante il gioco.

1.3. Glossario

Per facilitare la comprensione del documento, è stato creato un glossario che contiene i termini utilizzati nel documento e le loro definizioni. I termini presenti nel glossario sono colorati di blu e seguiti da un'asterisco: [esempio*](#).

Il glossario è accessibile tramite il link:

<https://github.com/fliposab/ProgettoStage/blob/main/Documentazione/Glossario.pdf>

oppure consultando il rispettivo documento all'interno della stessa cartella.

1.4. Riferimenti

1.4.1. Riferimenti normativi

- Piano di lavoro:

<https://github.com/fliposab/ProgettoStage/blob/main/Documentazione/Piano-di-lavoro.pdf>

1.4.2. Riferimenti informativi

- Glossario:

<https://github.com/fliposab/ProgettoStage/blob/main/Documentazione/Glossario.pdf>

2. Processi primari

2.1. Processo di fornitura

Il processo di fornitura è strutturato in conformità agli esiti previsti dalla clausola 5.2 dello [standard*](#) ISO/IEC 12207:1997. Tale processo include la definizione di requisiti concordati, l'analisi dei rischi associati, e la pianificazione di tempi e costi.

2.1.1. Scopo e descrizione

Il processo di fornitura ha lo scopo di garantire che il prodotto software sia consegnato al cliente in modo completo e funzionante.

2.1.2. Attività

Il processo di fornitura comprende le seguenti attività:

- Analisi dei requisiti:
raccolta e analisi dei requisiti del cliente, definizione delle specifiche del prodotto.
- Confronto:
confronto tra i requisiti del [relatore aziendale*](#) e le specifiche del prodotto, identificazione delle eventuali discrepanze.
- Pianificazione:
pianificazione delle attività di sviluppo, stima dei tempi e dei costi, definizione del piano di progetto.
- Progettazione:
definizione dell'architettura del sistema, progettazione dei componenti software.
- Sviluppo:
implementazione del codice sorgente, creazione dei test unitari.
- Verifica e validazione:
esecuzione dei test di integrazione, verifica della conformità alle specifiche.
- Consegna:
consegna del prodotto al cliente, formazione degli utenti, raccolta dei feedback.

2.1.3. Rapporto con il tutor aziendale

Il rapporto con il tutor aziendale è fondamentale per garantire che il prodotto software soddisfi le aspettative stabilite. Il tutor aziendale viene contattato in due modalità:

- Incontri periodici:
incontri settimanali per discutere lo stato di avanzamento del progetto, raccogliere feedback e risolvere eventuali problemi.

-
- Comunicazioni via email:
comunicazioni via email per condividere documenti, report di avanzamento e richieste di chiarimenti.

2.1.4. Documentazione

Il processo di fornitura deve essere documentato in modo dettagliato, in modo da garantire la tracciabilità delle attività svolte e delle decisioni prese. I documenti che il progetto presenta sono:

- **Analisi dei requisiti:** nel documento *Analisi dei requisiti* sono stati definiti tutti gli [use case*](#) e i requisiti funzionali del progetto. Questi sono stati raccolti in collaborazione con il tutor aziendale e sono stati utilizzati come base per la progettazione e lo sviluppo del software;
- **Piano di progetto:** il *Piano di progetto* è un documento che definisce le attività da svolgere e i tempi previsti per lo sviluppo del software, viene descritto in dettaglio ogni periodo di sviluppo, con una retrospettiva delle attività svolte e una pianificazione delle attività future;
- **Piano di qualifica:** nel documento *Piano di qualifica* sono definite le metriche che vengono usate per garantire la qualità del prodotto software. Vengono inoltre scritte le modalità di test e verifica del software, in modo da garantire che il prodotto soddisfi i requisiti stabiliti;
- **Specifiche tecnica:** la *Specifiche tecnica* è un documento che descrive in dettaglio l'architettura del sistema, i componenti software e le loro interazioni;
- **Manuale utente:** il *Manuale utente* è un documento che fornisce istruzioni dettagliate su come utilizzare il software e garantirne il corretto funzionamento.

2.2. Strumenti utilizzati

Gli strumenti utilizzati per lo sviluppo del software sono i seguenti:

- **Git:** sistema di controllo versione utilizzato per gestire il codice sorgente e le modifiche apportate.
- **GitHub:** piattaforma di hosting per il codice sorgente e la gestione del progetto, utilizzata per la collaborazione tra i membri del team di sviluppo.
- **Typst:** linguaggio di markup utilizzato per la composizione tipografica della documentazione.
- **Notion:** strumento di collaborazione e gestione dei progetti utilizzato per la pianificazione delle attività.

2.3. Sviluppo

2.3.1. Scopo e descrizione

Il processo di sviluppo è finalizzato alla realizzazione del software richiesto dal tutor, in base ai requisiti stabiliti durante i colloqui. Questo processo comprende tutte le attività necessarie

per la creazione del prodotto, dalla progettazione all'integrazione, garantendo che il prodotto creato soddisfi i requisiti concordati. Questo processo prevede le seguenti attività:

2.3.2. Attività

- **Analisi dei requisiti;**
- **Progettazione;**
- **Codifica;**
- **Modellazione;**
- **Testing;**

Ovviamente, il prodotto software atteso deve essere un software funzionante, che soddisfi i requisiti concordati e ampiamente testato. Sarà dunque necessario seguire i requisiti concordati seguendo le linee guida fissate.

2.3.2.1. Analisi dei requisiti

Lo sviluppo inizia con l'analisi dei requisiti, durante la quale vengono identificati tutti i casi d'uso e i requisiti dell'applicazione. Tutta questa attività è documentata nel documento «Analisi dei requisiti». I requisiti devono descrivere:

- funzionalità richieste;
- funzionalità di supporto;

I casi d'uso descrivono le interazioni tra il prodotto e gli attori coinvolti, ed aiutano ad individuare ulteriori requisiti. L'applicazione presenta un solo attore:

- giocatore.

Ogni use case ha la seguente struttura:

- diagramma del caso d'uso (se lo use case non è già presente in un altro diagramma);
- attori principali;
- attori secondari (se presenti);
- descrizione;
- precondizioni;
- postcondizioni;
- scenario principale;
- generalizzazioni, estensioni ed inclusioni, se presenti.

Per i requisiti, vi sono 3 tipi:

- funzionali;
- di qualità;
- di vincolo.

2.3.2.2. Progettazione

La progettazione è l'attività di definizione dell'architettura del software dal punto di vista logico. In questa fase si decide come soddisfare i requisiti identificati durante l'analisi.

In particolare, vengono definite le unità dell'applicazione e le loro interazioni, prestando attenzione a mantenerle separate e indipendenti per garantire una maggiore manutenibilità e scalabilità del sistema. Inoltre vanno definite le responsabilità che verranno applicate in fase di codifica assicurandosi di mantenere un livello di [efficienza*](#) e [efficacia*](#) il più alto possibile. L'approccio utilizzato in questa attività sarà sia [top-down*](#), per scomporre il problema in sotto-problemi, sia [bottom-up*](#), per ragionare sui singoli sotto-problemi e integrarli in una soluzione complessiva.

L'architettura prevista a fine attività sarà a monolite, dove tutti i componenti sono integrati e compilati in una singola applicazione.

2.3.2.3. Codifica

Obiettivo della fase di codifica è applicare le decisioni prese durante la fase di progettazione. Viene eseguita in parallelo con la fase di modellazione e consiste nella scrittura del codice sorgente secondo le specifiche definite nella fase di progettazione. Durante questa attività, si applicano le convenzioni di codifica stabilite e ogni componente viene sviluppato in modo modulare.

2.3.2.4. Modellazione

La fase di modellazione viene eseguita in parallelo con la fase di codifica. Comprende le seguenti sotto-attività:

- creazione ed esportazione dei modelli 3D;
- creazione delle animazioni;
- [baking*](#) delle textures.

I modelli vengono esportati nel formato [.glb*](#) e inseriti dentro la cartella *models*, assicurandosi che rispettino le specifiche tecniche e stilistiche definite in fase di progettazione.

2.3.2.5. Testing

Durante questa fase, vengono testati singolarmente i componenti e integrati progressivamente nel sistema complessivo. Vengono poi testate le prestazioni.

Eventuali problemi riscontrati vengono risolti tempestivamente, aggiornando la documentazione tecnica ove necessario.

2.3.3. Regole di sviluppo

Durante lo sviluppo, sono state imposte le seguenti regole per garantire omogeneità e correttezza. Le regole sono suddivise in base all'argomento:

- **Codifica:**

Tutti i file script del gioco sono salvati come file `.gd*`, e sono scritti con il linguaggio di programmazione `GDScript*`. I nomi delle classi sono salvate con una nomenclatura `PascalCase*`, mentre i nomi dei files e delle variabili usano `snake_case*`.

Per maggiori dettagli sulla nomenclatura, si seguono le convenzioni della documentazione ufficiale:

https://docs.godotengine.org/it/4.x/tutorials/scripting/gdscript/gdscript_styleguide.html

- **Modellazione:**

Tutti i modelli sono esportati nel formato `.glb*`. I `materiali*` vengono esportati senza immagini, dato che verranno rimpiazzati dal materiale presente nel gioco. Nel caso il modello presenti animazioni, vengono esportate insieme al modello.

- **Animazione:**

Le animazioni sono incluse nel modello durante l'esportazione. Per semplificare l'attività, viene usato un `rig*` che dispone di `Inverse Kinematics*`. Le animazioni sono già separate prima dell'esportazione e possono essere trovate nella sezione `NLA*` e selezionate individualmente premendo la linea con il mouse e modificarle usando la scorciatoia Shift+TAB.

- **Textures:**

Le textures sono salvate come semplici immagini di tipo `.png*`. Entrambe le dimensioni della texture (larghezza e altezza) devono essere una potenza di 2. Risoluzioni esempio:

- 256x256;
- 512x512;
- 1024x1024 (1K);
- 2048x2048 (2K).

Di norma, 1024 pixels corrispondono a 1 metro.

2.3.4. Strumenti usati

- **Visual Studio Code:** editor di testo utilizzato per la scrittura del codice sorgente e della documentazione.
- **Godot:** motore di gioco utilizzato per lo sviluppo del videogioco.
- **Blender:** software di modellazione 3D utilizzato per la creazione dei modelli, delle animazioni e delle texture del videogioco.

-
- **GIMP**: software di modifica delle immagini utilizzato per la modifica occasionale di [texture*](#) o e delle immagini del videogioco.

3. Processi di supporto

3.1. Documentazione

3.1.1. Scopo e descrizione

Il processo di documentazione ha lo scopo di garantire che tutta la documentazione necessaria per il progetto sia creata e mantenuta aggiornata.

In questa sezione verranno descritte le decisioni prese e rispettate, in modo da garantire la coerenza tra i vari documenti.

3.1.2. Attività

Il processo di documentazione comprende le seguenti attività:

- **Realizzazione:** creazione o modifica di un determinato documento, in base alle esigenze del progetto, si divide in:
 - **individuazione compito:** identificazione della sezione del documento da modificare o creare.
 - **redazione:** modifica del documento;
- **Revisione:** revisione delle modifiche applicate al documento, e decisione se approvarle o modificarle ulteriormente, comprende le seguenti attività:
 - **revisione interna:**
- **Aggiornamento:** comprende le seguenti attività:
 - **push:** i documenti vengono caricati nell [repository*](#) di [GitHub*](#)
 - **compilazione:** modifica dei documenti in base alle esigenze del progetto, in modo da garantire la coerenza tra i vari documenti.

3.1.3. Struttura dei documenti

Ci sono diversi tipi di documenti e generalmente sono organizzati nelle seguenti sezioni:

3.1.3.1. Intestazione

La prima pagina è l'intestazione del documento ed è composta generalmente dalle seguenti informazioni:

- **logo azienda:** logo dell'azienda ospitante;
- **logo università:** logo dell'università;
- **titolo:** titolo del documento e nome del file;
- **autore:** lo studente autore del documento;
- **data:** data dell'ultima modifica del documento, in DD/MM/YYYY.

3.1.3.2. Indice

La terza pagina, e se necessario le seguenti, è riservata all'indice del documento che elenca le sezioni di cui è composto il documento.

3.1.3.3. Corpo del documento

Il corpo del documento è strutturato in capitoli, ciascuno dei quali può essere suddiviso in sottocapitoli, che a loro volta possono contenere ulteriori suddivisioni.

3.1.4. Documenti del progetto

Verranno prodotti i seguenti documenti:

- [Norme di Progetto*](#);
- [Piano di Progetto*](#);
- [Analisi dei Requisiti*](#);
- [Piano di Qualifica*](#);
- [Manuale Utente*](#);
- [Specifica tecnica*](#);
- Glossario;

3.1.5. Elenchi puntati

Ogni voce di un elenco puntato finisce con «;», eccetto l'ultima che finisce con un punto.

3.1.6. Diagrammi Use Case

Per produrre i diagrammi uml degli use case, è stato utilizzato il seguente sito : app.diagrams.net. Successivamente vengono inseriti nei documenti opportuni come immagini.

3.1.7. Composizione tipografica

Per la composizione tipografica dei documenti si è deciso di usare [Typst*](#) per i seguenti motivi:

- semplicità degli strumenti utilizzati per la stesura;
- sintassi semplice;
- compilazione immediata;

Grazie a Typst si riesce facilmente a creare e mantenere un documento anche partendo da un template comune.

3.1.8. Strumenti

- **VS Code**: Editor di testo usato per scrivere i documenti;
- **Typst**: Linguaggio usato per la stesura dei documenti;
- **Github**: Servizio di hosting per il repository;

3.2. Gestione della configurazione

3.2.1. Scopo e descrizione

La gestione della configurazione è il processo utilizzato per mantenere software in un determinato stato coerente, con il fine di assicurare che il sistema funzioni nel modo previsto nonostante modifiche che potrebbero venir apportate durante il tempo. Possibili problemi nella configurazione potrebbero essere la causa di incoerenza o problemi di conformità che influenzano negativamente le operazioni.

3.2.2. Repository

La documentazione ed i file del progetto si possono trovare al seguente link:

<https://github.com/fliposab/ProgettoStage>.

Nella repository si possono trovare diverse cartelle che servono per organizzare e dividere i documenti ed il codice, in particolare:

- in **Documentazione** si trovano i documenti relativi al progetto;
- in **Sviluppo** si trovano i vari file usati per la creazione dell'applicazione.

Nella sezione «Release» inoltre è possibile scaricare l'MVP ed il PoC, disponibili con le seguenti versioni:

- v1.0-MVP: versione dell'MVP;
- v0.1-PoC: versione del PoC.

3.3. Gestione della qualità

3.3.1. Scopo e descrizione

Il processo di gestione della qualità ha come obiettivo quello di garantire che il software, la documentazione e tutto ciò che viene prodotto sia conforme ai requisiti di qualità specificati e richiesti. Gli obiettivi e gli standard di qualità richiesti e che devono essere soddisfatti sono indicati nel documento *Piano di qualifica*.

3.3.2. Attività

Seguendo lo standard ISO/IEC 12207:1995, le attività previste per questo processo sono:

- **implementazione:** in cui vengono realizzati gli strumenti necessari, per esempio i test, per poter attuare correttamente l'attività successiva di verifica, rivelando le varie problematiche e i metodi per mitigarle;
- **verifica:** in cui deve essere controllata l'efficacia dei processi, dei requisiti rispetto alla consistenza ed esaustività degli stessi, della documentazione prodotta, della progettazione rispetto ai requisiti individuati durante la fase di analisi e riportati nel documento *Analisi dei requisiti*, del codice e dell'integrazione delle varie componenti del sistema tra loro.

Quindi è l'attività che mette in pratica ciò che è stato individuato e realizzato durante l'attività di implementazione.

3.4. Verifica

3.4.1. Scopo e descrizione

Il processo di verifica ha lo scopo di garantire che il software sviluppato soddisfi i requisiti stabiliti e che sia conforme agli standard di qualità richiesti.

Viene eseguito durante tutto il processo di sviluppo, in modo da individuare eventuali problemi e risolverli tempestivamente.

3.4.2. Strumenti

- **GUT - Godot Unit Test:** add-on di Godot che permette di svolgere test di unità e di integrazione all'interno dell'editor;
- **Godot:** sono stati utilizzati gli strumenti per l'analisi delle prestazioni e risorse in uso forniti da Godot.

3.4.3. Tipi di verifica

Vi sono due tipologie di verifica, ognuna è focalizzata sulla verifica di vari aspetti dell'applicazione:

- **analisi statica;**
- **analisi dinamica.**

3.4.3.1. Analisi statica

L'analisi statica comporta il controllo del codice prima della sua esecuzione.

Questo tipo di verifica non viene applicata solo al codice, ma anche ai documenti del progetto.

Questo metodo viene applicato nei seguenti casi:

- individuazione di bug nel codice;
- individuazione di errori di battitura nei documenti;
- verifica della coerenza e completezza della documentazione prodotta;

3.4.3.2. Analisi dinamica

L'analisi dinamica viene eseguita all'esecuzione del software. Viene usata per controllare se ci sono errori durante l'esecuzione dell'applicazione e dei suoi componenti.

Questo tipo di verifica permette di individuare malfunzionamenti, errori logici o comportamenti inattesi che possono emergere solo durante l'esecuzione reale del software.

Le principali attività di analisi dinamica includono:

- esecuzione di test di unità e di integrazione per verificare il corretto funzionamento delle singole componenti e della loro interazione;
- monitoraggio delle prestazioni e dell'utilizzo delle risorse durante l'esecuzione;
- individuazione e correzione di bug che si manifestano solo in fase di runtime.

L'analisi dinamica è fondamentale per garantire che il software sia affidabile, efficiente e conforme ai requisiti stabiliti.

3.4.4. Denominazione dei test

Tutti i test svolti sono documentati nel documento *Piano di Qualifica*, suddivisi in base alla tipologia e ciò che svolgono.

I test sono identificati secondo la seguente nomenclatura:

T[Tipo]-[Identificativo]

dove:

- **Tipologia:** indica il tipo del test eseguito:
 - U: test di Unità;
 - I: test di Integrazione;
 - S: test di Sistema;
 - A: test di Accettazione;
- **Identificativo:** numero del test.

3.4.5. Verifica della documentazione

La verifica della documentazione è suddivisa nelle seguenti fasi:

- controllo ortografia e sintassi;
- controllo del rispetto delle norme tipografiche imposte;

-
- verifica della pertinenza dei contenuti scritti.

Per facilitare il controllo di errori ortografici è stato utilizzato un plugin su VS code che esegue uno spell check automatico su tutto il file.

3.5. Validazione

3.5.1. Scopo e descrizione

4. Processi organizzativi

4.1. Gestione dei processi

4.1.1. Scopo e descrizione

Il processo di gestione ha lo scopo di identificare le attività e i compiti da svolgere per proseguire nel progetto.

4.1.2. Attività

Tutti i processi organizzativi sono composti dalle seguenti attività

- **individuazione di un compito da svolgere:**
- **esecuzione:**
- **verifica:**

4.1.3. Strumenti usati

- **Github:** per gestire tutta la documentazione e il codice per il progetto in un repository;
- **Notion:** per la gestione delle attività e la pianificazione del progetto.

4.2. Miglioramento

4.2.1. Scopo e descrizione

Le attività di miglioramento hanno lo scopo di garantire che il processo di sviluppo del software sia continuamente migliorato e ottimizzato.

5. Metriche e standard per la qualità

Per migliorare e avere uno standard di qualità da cui attingere se è deciso di utilizzare degli standard riconosciuti a livello internazionale. Tra quelli disponibili è stato scelto lo standard ISO/IEC 12207:1995 per quanto riguarda la qualità dei processi principali, organizzativi e di supporto. Mentre per gli standard di qualità del software, lo standard ISO/IEC 25010:2023. Le principali caratteristiche prese in considerazione sono:

- **Funzionalità;**
- **Performance;**
- **Affidabilità;**
- **Usabilità;**
- **Portabilità;**
- **Manutenibilità;**

5.1. Funzionalità

La funzionalità riguarda e misura il grado di soddisfacibilità dei requisiti rispetto al prodotto finale. In particolare vengono misurate:

- **Completezza:** il software prodotto deve soddisfare tutti i requisiti obbligatori e le funzionalità previste;
- **Correttezza:** il funzionamento del software deve rispettare le specifiche dichiarate;
- **Adeguatezza:** il software prodotto deve essere idoneo allo scopo per cui è stato pensato e al contesto in cui viene usato;
- **Conformità:** il software prodotto deve rispettare le norme predefinite e il grado di qualità previsto;

5.2. Performance

La performance misura l'efficienza di un prodotto e la capacità del prodotto stesso di gestire le risorse in modo direttamente proporzionale alle prestazioni che vengono fornite. In particolare vengono misurate:

- **Risorse:** l'uso delle risorse deve essere efficiente;
- **Tempo:** il software prodotto deve dare una risposta in tempi consoni alle richieste che gli vengono fatte;
- **Conformità:** il prodotto deve rispettare dei vincoli di qualità;
- **Capacità:** il prodotto deve saper gestire i carichi di lavoro attesi senza compromettere le prestazioni previste;

5.3. Affidabilità

L'affidabilità è il parametro per misurare se un prodotto reagisce ai problemi senza compromettere le prestazioni. In particolare vengono misurate:

- **Tolleranza ai guasti e agli errori:** il prodotto deve saper gestire gli errori e rispondere a malfunzionamenti o guasti senza che si causino interruzioni gravi nel prodotto e allo stesso tempo mantenere un certo grado di prestazioni;
- **Maturità:** il prodotto deve garantire affidabilità e stabilità cercando di evitare che si vengano a generare errori o malfunzionamenti;
- **Recuperabilità:** a seguito di un qualsiasi tipo di errore, guasto o malfunzionamento il prodotto deve essere in grado di tornare alle sue prestazioni standard ripristinando le sue funzionalità;
- **Disponibilità:** il prodotto deve essere accessibile e correttamente funzionante, quindi operativo ogni qual volta sia necessario;
- **Aderenza:** il prodotto deve rispettare gli standard di qualità prefissati;

5.4. Sicurezza

La sicurezza di un prodotto misura il suo grado di protezione da minacce e vulnerabilità, garantendo che i dati e le funzionalità rimangano integri, disponibili e riservati.

- **Riservatezza:** il software deve garantire la protezione dei dati sensibili;
- **Integrità:** il software deve garantire che i dati siano completi, accurati e sicuri;
- **Autenticazione:** il software deve controllare e verificare le credenziali degli utenti e limitare l'accesso ai soli utenti autorizzati;
- **Autenticità:** il prodotto deve permettere di verificare la provenienza dei dati;

5.5. Usabilità

L'usabilità ci permette di calcolare e comprendere quando e in quanto tempo l'utente finale riesca ad apprendere le modalità di utilizzo di un prodotto. In particolare vengono misurate:

- **Apprendibilità:** la semplicità con cui l'utente riesce ad apprendere le funzionalità del prodotto;
- **Comprensibilità:** la facilità con cui l'utente comprende il funzionamento del prodotto e riesce ad utilizzarlo in modo appropriato;
- **Riconoscibilità:** il prodotto deve fornire un'interfaccia utente intuitiva e di facile comprensione;
- **Estetica:** l'interfaccia del prodotto deve essere gradevole;
- **Operabilità:** quanto è semplice per l'utente usare il prodotto in maniera corretta;

5.6. Portabilità

La portabilità è la capacità di un prodotto di essere facilmente spostato da un ambiente di esecuzione ad un altro senza insorgere in problemi. In particolare vengono misurate:

- **Adattabilità:** il software deve essere capace di funzionare in ambienti di esecuzione diversi in modo corretto;
- **Installabilità:** il software deve poter essere installato e configurato facilmente e rapidamente;
- **Sostituibilità:** il prodotto deve poter funzionare correttamente nel momento in cui venga aggiornato e quindi sostituito da nuove versioni;

5.7. Manutenibilità

La manutenibilità di un prodotto misura la facilità con cui può essere modificato, corretto e migliorato nel tempo. In particolare vengono misurate:

- **Analizzabilità:** la facilità con cui il codice può essere controllato al fine di risolvere eventuali errori e problemi;
- **Modificabilità:** il software deve poter essere modificato e migliorato in maniera semplice;
- **Testabilità:** la semplicità con cui il prodotto può essere testato;
- **Riutilizzabilità:** le varie parti del software devono poter essere riutilizzate in progetti o ambienti differenti;
- **Stabilità:** capacità del prodotto di continuare a funzionare senza gravi problemi a seguito di modifiche sbagliate;